Algorithm Design

Midterm Review

Hengxi Liu
Email: liu-hx26@mails.tsinghua.edu.cn

Codex (gpt-6.1-sol)

Review map

1. **Introduction:** stable matching and Gale–Shapley.
2. **Graph algorithms:** DFS, BFS, Kosaraju, Dijkstra, MST (Kruskal, Prim).
3. **Greedy:** interval scheduling, exchange arguments, Huffman codes.
4. **Dynamic programming:** subproblems, recurrences, Bellman–Ford, treewidth.
5. **Intractability:** P, NP, reductions, NP-hardness, NP-completeness.

What to be able to explain

What is the problem? Why does the algorithm work? How long does it take?

Pseudocode conventions: values and shared state

Parameter passing follows CLRS

Parameters are passed by value. For an array or object, the copied value is a pointer to the same underlying data.

- Updating $A[i]$ or $u.color$ is visible to the caller; rebinding a local parameter to another object is not.
- Queue, heap, Union–Find, and engagement operations modify their shared data structures in place.
- DFS uses one global *time* counter for the whole traversal; vertex attributes are shared across recursive calls.
- MEMO-OPT receives a shared cache M, initialized once before the first call.
- A heap's key array is shared: DECREASE-KEY updates that array and restores heap order as one operation.

CLRS, *Introduction to Algorithms*, 3rd ed., §2.1, p.21.

Stable matching: the definition

Two sets M, W, each of size n; everyone ranks all members of the other set, without ties.

- A **matching** pairs each participant with at most one partner.
- A **perfect matching** pairs every participant exactly once.
- An **instability** is an unmatched pair whose members each prefer the other to their current partner.

> "We'll say that a matching S is stable if (i) it is perfect, and (ii) there is no instability with respect to S."

An instability is also called a **blocking pair**.

Kleinberg–Tardos, *Algorithm Design*, §1.1, p.4.

Gale–Shapley: deferred acceptance

GALE-SHAPLEY(M, W)

```
1  mark all participants free
2  while a free man m has an unproposed woman
3      w = highest-ranked such woman on m's list
4      record that m has proposed to w
5      if w is free
6          ENGAGE(m, w)
7      else if w prefers m to partner[w]
8          m' = partner[w]
9          partner[m'] = NIL; mark m' free
10         ENGAGE(m, w)
11 return the engaged pairs
```

ENGAGE(m, w)

Update their shared partner records and mark both engaged.

- A rejected man remains free.
- Proposals move down each man's list.
- An engaged woman's partner only improves.

Kleinberg–Tardos, *Algorithm Design*, §1.1, pp.6–7.

Why Gale–Shapley works

Termination. Each pair receives at most one proposal: at most n^2 proposals.

Perfect matching. If a man were free at termination, he would have proposed to every woman. Every woman would then be engaged, contradicting that a man is free.

Stability. Suppose m prefers w to his final partner. He must have proposed to w earlier. She rejected him, or later replaced him, for someone she preferred. Her final partner is at least as good; (m, w) cannot block.

Running time

$O(n^2)$ with preference arrays, inverse ranks, and a list of free men.

Kleinberg–Tardos, *Algorithm Design*, §1.1, pp.7–9; §2.3, pp.45–47.

Exercise 1: Hospital–resident matching

There are m hospitals with capacities c_h and $n > \sum_h c_h$ students. Hospitals rank all students; students rank all hospitals, without ties.

Assignment requirements

Assign each student to at most one hospital, filling every hospital's capacity. Require stability: no hospital prefers an unassigned student to one of its residents; no assigned student prefers another hospital that also prefers this student to a current resident.

Design an algorithm that always finds such an assignment. Prove termination and stability, and analyze the running time.

Kleinberg–Tardos, *Algorithm Design*, Ch.1, Ex.4, pp.23–24.

Solution 1: Hospital–resident matching

```
while some free student s has an untried hospital
    h ← next hospital on s's preference list
    tentatively assign s to h
    if h now holds more than c_h students
        reject h's least-preferred held student
return the tentative assignments
```

Termination and capacity. Each student proposes to each hospital at most once. If a position were empty at termination, some unmatched student would have proposed there and could not have been rejected; hence every position is filled.

Stability. A student who prefers h to the final outcome proposed to h earlier. After rejection, h always holds c_h students it prefers to that student, so this pair cannot block.

Cost. At most nm proposals. Precompute ranks and use a max-heap per hospital: $O(nm \log(n+1))$ time, including input processing.

Kleinberg–Tardos, *Algorithm Design*, Ch.1, Ex.4, pp.23–24.

Depth-first search: DFS and DFS-VISIT

DFS(G)

```
1  for each vertex u ∈ G.V
2      u.color = WHITE
3      u.π = NIL
4  time = 0
5  for each vertex u ∈ G.V
6      if u.color = WHITE
7          DFS-VISIT(G, u)
```

DFS-VISIT(G, u)

```
1   time = time + 1
2   u.d = time
3   u.color = GRAY
4   for each v ∈ G.Adj[u]
5       if v.color = WHITE
6           v.π = u
7           DFS-VISIT(G, v)
8   u.color = BLACK
9   time = time + 1
10  u.f = time
```

- *time* is global; $u.d$ and $u.f$ are discovery and finishing times.
- WHITE: undiscovered; GRAY: active; BLACK: finished. Always $u.d < u.f$.
- The outer loop covers every component. Total time: $\Theta(|V| + |E|)$.

CLRS, *Introduction to Algorithms*, 3rd ed., §22.3, pp.604–605.

Breadth-first search: BFS

BFS(G, s): initialize

```
1  for each vertex u ∈ G.V \ {s}
2      u.color = WHITE
3      u.d = ∞
4      u.π = NIL
5  s.color = GRAY
6  s.d = 0
7  s.π = NIL
8  Q = ∅
9  ENQUEUE(Q, s)
```

BFS(G, s): continue

```
10 while Q ≠ ∅
11     u = DEQUEUE(Q)
12     for each v ∈ G.Adj[u]
13         if v.color = WHITE
14             v.color = GRAY
15             v.d = u.d + 1
16             v.π = u
17             ENQUEUE(Q, v)
18     u.color = BLACK
```

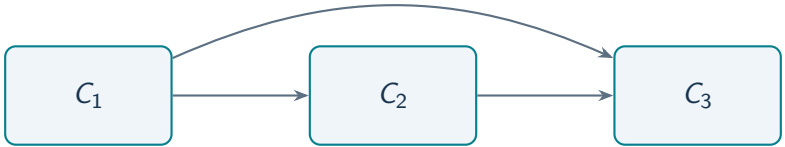
- $v.d$ is the shortest distance from s in edges; $v.\pi$ is its predecessor.
- Q is a FIFO queue of GRAY vertices; each vertex is enqueued at most once.
- Unreachable vertices retain $d = \infty$. Time: $\Theta(|V| + |E|)$.

CLRS, *Introduction to Algorithms*, 3rd ed., §22.2, pp.595–597.

Strongly connected components

"a directed graph is strongly connected if, for every two nodes u and v, there is a path from u to v and a path from v to u."

An **SCC** is a maximal set of mutually reachable vertices.



- SCCs partition the vertices.
- Contracting each SCC produces a **DAG**.
- Reversing all edges preserves the SCCs.

Kleinberg–Tardos, *Algorithm Design*, §3.5, pp.98–99; CS161, Winter 2011 Handout #16, pp.3–4.

Kosaraju: two passes of DFS

KOSARAJU(G)

```
1  G^T = REVERSE-EDGES(G)
2  DFS(G^T)
3  L = vertices in decreasing finishing-time order from G^T
4  mark all vertices of G unexplored
5  C = ∅
6  for each v in L
7      if v is unexplored
8          C = DFS-COLLECT(G, v)
9          C = C ∪ {C}
10 return C
```

- DFS-COLLECT marks vertices explored and returns the vertices newly reached by that call; marks persist between calls.
- Build L by reversing the first pass's finishing list: no comparison sort.
- Edge reversal and both DFS passes take $O(|V| + |E|)$ time in total.

CS161, Winter 2011 Handout #16, pp.1–3.

Why the finishing-time order matters

Key lemma: first pass on G^T

If an edge of G goes from SCC C_1 to a distinct SCC C_2, then

$$\max_{v \in C_1} f(v) < \max_{v \in C_2} f(v).$$



- The largest remaining finishing time lies in a sink SCC of the remaining graph G.
- DFS reaches that whole SCC and cannot enter another unexplored SCC.
- Remove it conceptually and repeat the argument.

CS161, Winter 2011 Handout #16, pp.3–5.

Dijkstra: single-source shortest paths

DIJKSTRA(G, ℓ, s)

```
1  for each v ∈ V
2      d[v] = ∞;  π[v] = NIL
3  d[s] = 0;  S = ∅
4  Q = MIN-HEAP(V, key d)
5  while Q ≠ ∅
6      u = EXTRACT-MIN(Q)
7      if d[u] = ∞: break
8      S = S ∪ {u}
9      for each v ∈ G.Adj[u]
10         if v ∉ S and d[v] > d[u] + ℓ_uv
11             π[v] = u
12             DECREASE-KEY(Q, v, d[u] + ℓ_uv)
13 return (d, π)
```

Assumption

$\ell_e \geq 0$ for every edge.

S: vertices with finalized distances.

After processing s, the next finite distance selected is

$$d'(v) = \min_{\substack{(u,v)\in E \\ u\in S}} \{d[u] + \ell_{uv}\}.$$

π records a shortest-path tree.

Kleinberg–Tardos, *Algorithm Design*, §4.4, pp.137–142.

Dijkstra: correctness and implementation

Invariant

For every $u \in S$, $d(u)$ is the true shortest-path distance from s to u.

First extract s: $d[s] = 0$ is correct. Thereafter $s \in S$. Suppose the algorithm next selects a vertex v with finite distance. Take any s–v path P and its first edge (x, y) leaving S.

$$\ell(P) \;\geq\; d(x) + \ell_{xy} \;\geq\; d'(y) \;\geq\; d'(v).$$

The first inequality uses the induction hypothesis and nonnegative remaining edge lengths. A path of length $d'(v)$ exists, so it is shortest.

- Priority queue: repeatedly extract the smallest tentative distance and relax outgoing edges.
- Binary heap: $O((n+m)\log n)$; the book states $O(m \log n)$ assuming every vertex is reachable from s.
- Unreachable vertices retain distance ∞.

Kleinberg–Tardos, *Algorithm Design*, §4.4, pp.139–142.

Minimum spanning tree (MST)

Input: a connected, undirected graph $G = (V, E)$ with edge weights ℓ_e.

"We will call a subset $T \subseteq E$ a spanning tree of G if (V, T) is a tree."

Objective

Find a spanning tree minimizing

$$w(T) = \sum_{e \in T} \ell_e.$$

- A spanning tree connects all n vertices, has no cycles, and has $n - 1$ edges.
- Kruskal grows a forest; Prim grows one tree from a root.
- Equal or negative edge weights are allowed. An MST need not be unique.
- For a disconnected graph, the corresponding goal is a minimum spanning forest.

Kleinberg–Tardos, *Algorithm Design*, §4.5, pp.142–149.

MST correctness: the cut property

A cut $(S, V \setminus S)$ **respects** a chosen edge set A if no edge of A crosses the cut.

Safe-edge property, allowing ties

Suppose A is contained in some MST and the cut respects A.
If e is a minimum-weight edge crossing the cut, then $A \cup \{e\}$ is contained in some MST.

Exchange argument

1. Take an MST T containing A. If $e \in T$, we are done.
2. Adding e creates a cycle. The path in T joining its endpoints contains another cut-crossing edge f.
3. Since the cut respects A, $f \notin A$, and $\ell_e \leq \ell_f$.
4. $T' = T - \{f\} + \{e\}$ is a spanning tree containing $A \cup \{e\}$, with $w(T') \leq w(T)$; it is also an MST.

Kleinberg–Tardos, *Algorithm Design*, §4.5, pp.145–149.

Kruskal: process edges in increasing weight order

KRUSKAL(G, ℓ)

```
1  T = ∅
2  for each v ∈ V
3      MAKE-SET(v)
4  sort E by nondecreasing weight
5  for each edge (u, v) in sorted order
6      if FIND(u) ≠ FIND(v)
7          T = T ∪ {(u, v)}
8          UNION(u, v)
9          if |T| = |V| − 1: break
10 return T
```

Union–Find invariant

The disjoint sets are exactly the connected components of (V, T).

Adding an edge between different components cannot create a cycle.

Safe cut: one current component. Each accepted edge is a cheapest edge leaving it.

Time: $O(m \log m) = O(m \log n)$ for a simple connected graph; sorting dominates with efficient Union–Find.

Kleinberg–Tardos, *Algorithm Design*, §4.5, pp.143–146; §4.6, pp.151–157.

Prim: grow one tree from a root

PRIM(G, ℓ, r)

```
1  for each v ∈ V
2      key[v] = ∞;  π[v] = NIL
3  key[r] = 0;  T = ∅
4  Q = MIN-HEAP(V, key key)
5  while Q ≠ ∅
6      u = EXTRACT-MIN(Q)
7      if π[u] ≠ NIL
8          T = T ∪ {(π[u], u)}
9      for each v ∈ G.Adj[u]
10         if v ∈ Q and ℓ_uv < key[v]
11             π[v] = u
12             DECREASE-KEY(Q, v, ℓ_uv)
13 return T
```

After extracting r, let S be the extracted vertices.

Attachment cost

For $v \notin S$,

$$key[v] = \min_{\substack{(u,v)\in E \\ u\in S}} \ell_{uv}.$$

$\pi[v]$ records the attaching vertex.

Safe cut: $(S, V \setminus S)$. Every added edge is cheapest across this cut.

Kleinberg–Tardos, *Algorithm Design*, §4.5, pp.143–150.

Kruskal, Prim, and Dijkstra: distinguish the choices

	Kruskal	**Prim**
Partial solution	A forest of components	One growing tree
Next choice	Cheapest edge joining different components	Cheapest edge leaving the current tree
Main data structure	Sorted edges + Union–Find	Min-priority queue of vertices
Time, simple connected graph	$O(m \log n)$	$O(m \log n)$ with a binary heap

Prim versus Dijkstra: similar priority-queue structure, different updates.

$$\text{Prim:} \quad key[v] \leftarrow \min\{key[v], \ell_{uv}\},$$
$$\text{Dijkstra:} \quad d[v] \leftarrow \min\{d[v], d[u] + \ell_{uv}\}.$$

Prim minimizes total tree weight; Dijkstra computes distances from a source.

Kleinberg–Tardos, *Algorithm Design*, §4.4–4.6, pp.137–157.

Exercise 2: Semiconnected directed graphs

A directed graph is **semiconnected** if for every pair u, v, either u can reach v or v can reach u. Design an efficient algorithm deciding whether a directed graph is semiconnected. Prove both directions of your criterion and analyze the running time. You may use SCC computation and topological sorting as subroutines. A topological ordering of a DAG places the tail of every edge before its head.

CLRS, *Introduction to Algorithms*, 3rd ed., Ex.22.5-7, p.621.

Solution 2: Semiconnected directed graphs

```
compute the SCC condensation DAG H
C_1, ..., C_k ← a topological ordering of H
for i = 1, ..., k − 1
    if (C_i, C_{i+1}) ∉ E(H): return false
return true
```

If all consecutive edges exist: the chain $C_1 \to \cdots \to C_k$ gives reachability between every pair of components. Vertices within an SCC are mutually reachable.

If G is semiconnected: consecutive components must be comparable. Topological order rules out $C_{i+1} \rsquigarrow C_i$. A path $C_i \rsquigarrow C_{i+1}$ cannot have an intermediate component, so the direct edge must exist.

Cost. $O(|V| + |E|)$ time and space. Scan condensation edges once, marking whether each required consecutive edge occurs.

CLRS, *Introduction to Algorithms*, 3rd ed., Ex.22.5-7, p.621.

Exercise 3: Second-best minimum spanning tree (structure)

Let G be connected and undirected, with $m \geq n$ and all edge weights distinct. Write $\mathcal{T}$ for its spanning trees and T^* for an MST.

$$T^{(2)} \in \arg \min_{T \in \mathcal{T} \setminus \{T^*\}} w(T).$$

1. Prove that the MST is unique, but a second-best MST need not be unique.
2. Prove that some second-best tree can be obtained from T^* by deleting one edge and inserting one edge outside T^*.

CLRS, *Introduction to Algorithms*, 3rd ed., Problem 23-1(a)–(b), p.638.

Exercise 3: Second-best minimum spanning tree (algorithm)

Continue with the distinct-weight graph and MST T^* from the preceding part.

1. Given any spanning tree T, compute, for all pairs u, v, a maximum-weight edge on the unique u–v path in T. The preprocessing time must be $O(n^2)$.
2. Use the one-edge-exchange property and this preprocessing to compute a second-best MST efficiently. Prove correctness and analyze the total running time.

Your algorithm must output a tree, not just the weight of the second-best solution.

CLRS, *Introduction to Algorithms*, 3rd ed., Problem 23-1(c)–(d), p.638.

Solution 3: Second-best minimum spanning tree

Structure. Distinct weights imply a unique MST T: exchanging the lightest edge in the symmetric difference of two MSTs contradicts optimality. For a second-best tree U, choose $e \in U \setminus T$. The T-path between its endpoints crosses the cut of $U - e$ at some $f \in T \setminus U$. The MST cycle property gives $w(f) < w(e)$. Thus $U - e + f$ is cheaper than U and must equal T.

Algorithm. Run Kruskal to find T. From each vertex, traverse T, propagating the heaviest edge seen so far; store $M[u, v]$ for every pair. Choose

$$e^* = (u^*, v^*) \in \arg \min_{(u,v) \notin T} \{w(u, v) - w(M[u, v])\}, \qquad \text{return } T - \{M[u^*, v^*]\} + \{e^*\}.$$

Every candidate is a spanning tree, and the one-exchange property ensures that a second-best tree is among these candidates. Time $O(m \log n + n^2)$; space $O(n^2)$.

Second-best need not be unique. Path edges ab, bc, cd have weights $1, 2, 3$; add ac of weight 4 and bd of weight 5. The MST costs 6; swapping bc for ac, or cd for bd, gives two second-best trees of cost 8.

CLRS, *Introduction to Algorithms*, 3rd ed., Problem 23-1(a)–(d), p.638.

What makes an algorithm greedy?

"An algorithm is greedy if it builds up a solution in small steps, choosing a decision at each step myopically to optimize some underlying criterion."

- Specify the local choice rule precisely.
- Show the resulting solution is feasible.
- Prove that the local choices preserve global optimality.

Two recurring proof techniques

Stays ahead: compare the progress of partial solutions.
Exchange argument: transform a competing solution without worsening it.

Kleinberg–Tardos, *Algorithm Design*, Chapter 4, pp.115–116.

Interval scheduling: finish as early as possible

Problem: choose a maximum-cardinality subset of nonoverlapping intervals.

INTERVAL-SCHEDULE(R)

```
1 sort R by nondecreasing finish time
2 A = ∅; lastFinish = −∞
3 for each interval i in sorted order
4     if s_i ≥ lastFinish
5         A = A ∪ {i}
6         lastFinish = f_i
7 return A
```

"we should accept first the request that finishes first, that is, the request i for which $f(i)$ is as small as possible."

Time: $O(n \log n)$ including sorting; $O(n)$ for the scan.

Kleinberg–Tardos, *Algorithm Design*, §4.1, pp.116–121.

Interval scheduling: the stays-ahead proof

Let greedy choose $i_1, \ldots, i_k$ and an optimum choose $j_1, \ldots, j_m$, both in time order.

Claim

$f(i_r) \leq f(j_r)$ whenever both r-th intervals exist.

- **Base:** i_1 has minimum finish time.
- **Induction:** $f(i_{r-1}) \leq f(j_{r-1}) \leq s(j_r)$, so j_r is available when greedy chooses i_r.
- Greedy chooses the available interval with minimum finish time, hence $f(i_r) \leq f(j_r)$.

If $m > k$, then j_{k+1} is compatible with all greedy choices, contradicting that greedy stopped. Thus $k = m$.

Kleinberg–Tardos, *Algorithm Design*, §4.1, pp.119–121.

Exchange arguments: the proof pattern

"one considers any possible solution to the problem and gradually transforms it into the solution found by the greedy algorithm without hurting its quality."

1. Start with an optimal solution O.
2. Find a disagreement with the greedy solution.
3. Exchange elements or reorder choices to remove that disagreement.
4. Prove feasibility and show the objective does not worsen.
5. Show progress, then repeat until the solutions agree.

Kleinberg–Tardos, *Algorithm Design*, Chapter 4, p.116; §4.2, pp.128–130.

Exchange example: minimizing maximum lateness

Jobs have processing times t_i and deadlines d_i. Schedule all jobs on one machine, all available at time 0.

$$L = \max_i \max\{0, C_i - d_i\}.$$

Greedy rule: schedule by nondecreasing deadline, without idle time.

- Take adjacent inverted jobs: i before j but $d_i > d_j$.
- Swap them. Job j finishes earlier.
- After the swap, $C_i' = C_j$, so

$$C_i' - d_i = C_j - d_i \leq C_j - d_j.$$

- Other completion times do not change; maximum lateness cannot increase.

Each swap removes an inversion. Eventually the optimal schedule is in deadline order.

Kleinberg–Tardos, *Algorithm Design*, §4.2, pp.125–130.

Huffman coding: the optimization problem

"a prefix code for a set S of letters is a function γ that maps each letter $x \in S$ to some sequence of zeros and ones, in such a way that for distinct $x, y \in S$, the sequence $\gamma(x)$ is not a prefix of the sequence $\gamma(y)$."

Given letter frequencies f_x, minimize the average bits per letter:

$$\mathrm{ABL}(\gamma) = \sum_{x \in S} f_x |\gamma(x)|, \qquad \sum_{x \in S} f_x = 1.$$

In a binary code tree, letters are leaves and codeword lengths are leaf depths.

Kleinberg–Tardos, *Algorithm Design*, §4.8, pp.164–168.

Huffman: repeatedly merge the two smallest frequencies

HUFFMAN(S, f)

```
1  Q = MIN-HEAP(one leaf for each x ∈ S, key f_x)
2  while |Q| > 1
3      x = EXTRACT-MIN(Q)
4      y = EXTRACT-MIN(Q)
5      z = NEW-NODE()
6      z.left = x ; z.right = y
7      f_z = f_x + f_y
8      INSERT(Q, z)
9  return EXTRACT-MIN(Q)
```

- The returned node is the root of the code tree.
- Label left/right edges 0/1; root-to-leaf paths give the codewords.
- For k letters, $k - 1$ merges take $O(k \log k)$ time with a binary heap.
- Frequency ties may be broken arbitrarily.

Kleinberg–Tardos, *Algorithm Design*, §4.8, pp.171–175.

Why Huffman's greedy choice is safe

"There is an optimal prefix code, with corresponding tree T^*, in which the two lowest-frequency letters are assigned to leaves that are siblings in T^*."

- **Exchange:** move the two lowest-frequency letters to deepest sibling leaves without increasing average length.
- **Contract:** replace the siblings by a meta-letter of combined frequency f_ω.
- **Recurse:** optimally code the smaller alphabet, then expand the siblings.

$$\mathrm{ABL}(T) = \mathrm{ABL}(T') + f_\omega.$$

A min-heap gives $O(k \log k)$ time for k letters.

Kleinberg–Tardos, *Algorithm Design*, §4.8, pp.170–175.

Exercise 4: Weighted completion time

A single machine must process all n jobs, one at a time. Job i takes time $t_i > 0$ and has weight $w_i > 0$. Let C_i be its completion time.

$$\text{Minimize} \quad \sum_{i=1}^{n} w_i C_i.$$

Derive an efficient ordering rule and explain how ties are handled. Prove optimality using an exchange argument and analyze the running time.

Kleinberg–Tardos, *Algorithm Design*, Ch.4, Ex.13, pp.194–195.

Solution 4: Weighted completion time

Rule. Process jobs in nondecreasing order of t_i/w_i, with no idle time. Break equal ratios arbitrarily. Compare $t_i w_j$ and $t_j w_i$ to avoid division.

Adjacent exchange. For adjacent jobs i, j, starting at time T, all other completion times are unchanged by swapping. The difference is

$$\begin{aligned}\text{cost}(ij) - \text{cost}(ji) &= w_i(T + t_i) + w_j(T + t_i + t_j) \\ &\quad - w_j(T + t_j) - w_i(T + t_j + t_i) \\ &= w_j t_i - w_i t_j.\end{aligned}$$

If $t_i/w_i > t_j/w_j$, swapping strictly decreases the objective. Repeatedly remove inversions to obtain the proposed order; equal ratios change nothing.

Cost. Sorting takes $O(n \log n)$ time; outputting the schedule takes $O(n)$ time.

Kleinberg–Tardos, *Algorithm Design*, Ch.4, Ex.13, pp.194–195.

Exercise 5: Ternary Huffman codes

Generalize Huffman's algorithm from binary codewords to codewords over $\{0, 1, 2\}$. Give an algorithm for an arbitrary finite alphabet with positive symbol frequencies, including initialization and base cases for every alphabet size.

Prove that the average codeword length is minimum among all ternary prefix codes and analyze the running time.

CLRS, *Introduction to Algorithms*, 3rd ed., Ex.16.3-7, p.436.

Solution 5: Ternary Huffman codes

```
if the alphabet has one symbol: give it the empty codeword
otherwise
    if the alphabet size is even: add a zero-frequency dummy
    Q ← min-heap of symbol leaves, keyed by frequency
    while |Q| > 1
        remove the three minimum-frequency trees a, b, c
        insert their parent with frequency f(a) + f(b) + f(c)
    label child edges 0, 1, 2; read root-to-leaf codewords
    discard the dummy symbol's codeword, if any
```

Why padding? A full ternary tree has an odd number of leaves. Any optimal tree can be completed with zero-frequency dummy leaves; for even alphabet size one dummy suffices (an optimal tree can have just one binary internal node).

Greedy step. In an optimal full tree, exchange the three smallest frequencies into deepest sibling leaves. Contract these siblings; expansion adds exactly their frequency sum to the cost. Induction proves optimality, including the padded case. Heap operations take $O(k \log k)$ time and $O(k)$ space.

CLRS, *Introduction to Algorithms*, 3rd ed., Ex.16.3-7, p.436.

Dynamic programming: build solutions from subproblems

"one implicitly explores the space of all possible solutions, by carefully decomposing things into a series of subproblems, and then building up correct solutions to larger and larger subproblems."

- Identify a reusable family of subproblems.
- Express each answer using answers to smaller subproblems.
- Compute and save each needed answer once.

A useful contrast

Greedy commits to a local choice. DP compares alternatives through a recurrence.

Kleinberg–Tardos, *Algorithm Design*, Chapter 6, p.251; §6.1–6.2, pp.254–260.

A checklist for DP solutions

1. **State:** what exactly does each table entry mean?
2. **Recurrence:** split an optimal solution into exhaustive cases.
3. **Base cases:** specify empty or smallest instances.
4. **Order:** ensure dependencies are already computed.
5. **Answer and reconstruction:** identify the answer entry and trace choices if a solution is required.
6. **Complexity:** count states and work per state.

For a polynomial-time DP

Both the number of states and the work per state must be polynomial in the input length.

Kleinberg–Tardos, *Algorithm Design*, §6.2, pp.258–260.

Weighted interval scheduling: derive the recurrence

Sort by finish time: $f_1 \le \cdots \le f_n$. Interval j has value v_j.

- $p(j)$: largest $i < j$ with $f_i \le s_j$, or 0 if none exists.
- $\mathrm{OPT}(j)$: maximum value using intervals $1, \ldots, j$.
- An optimal solution either includes j or excludes j.

Recurrence

$$\mathrm{OPT}(j) = \max\{v_j + \mathrm{OPT}(p(j)),\ \mathrm{OPT}(j-1)\}, \qquad \mathrm{OPT}(0) = 0.$$

If j is included, the remaining intervals must optimally solve the compatible prefix. Otherwise we could improve the whole solution.

Kleinberg–Tardos, *Algorithm Design*, §6.1, pp.253–254.

Memoization and iteration compute the same recurrence

MEMO-OPT(j, M)

Initialize the shared cache once: $M[0] = 0$; other entries UNDEFINED.

```
1 if M[j] ≠ UNDEFINED
2     return M[j]
3 take = v_j + MEMO-OPT(p(j), M)
4 skip = MEMO-OPT(j − 1, M)
5 M[j] = max{take, skip}
6 return M[j]
```

Call MEMO-OPT(n, M).

ITERATIVE-OPT(n)

Evaluate in increasing prefix order.

```
1 M[0] = 0
2 for j = 1 to n
3     take = v_j + M[p(j)]
4     skip = M[j − 1]
5     M[j] = max{take, skip}
6 return M[n]
```

Cost: $n + 1$ states and $O(1)$ work per state give $O(n)$ time after preprocessing. Sorting and binary searches for $p(j)$ give $O(n \log n)$ overall.

Kleinberg–Tardos, *Algorithm Design*, §6.1–6.2, pp.256–259.

0/1 knapsack: add a capacity dimension

Item i has nonnegative integer weight w_i and value v_i. Maximize value subject to total weight at most W; use each item at most once.

$$\mathrm{OPT}(i, w) = \text{maximum value using items } 1, \ldots, i \text{ with capacity } w.$$

Recurrence

$$\mathrm{OPT}(i, w) = \begin{cases} \mathrm{OPT}(i-1, w), & w_i > w, \\ \max\{\mathrm{OPT}(i-1, w),\ v_i + \mathrm{OPT}(i-1, w-w_i)\}, & w_i \le w. \end{cases}$$

Base: $\mathrm{OPT}(0, w) = 0$ for every $w \ge 0$.

$O(nW)$ time; $O(nW)$ space for the full table. Both branches use row $i-1$, preventing reuse of item i.

Kleinberg–Tardos, *Algorithm Design*, §6.4, pp.267–272.

Pseudo-polynomial time is not polynomial in input length

"it is a polynomial function of n and W, the largest integer involved in defining the problem. We call such algorithms pseudo-polynomial."

- The knapsack and Subset Sum DPs use $O(nW)$ time.
- A binary integer W needs only $\Theta(\log W)$ bits.
- If $W = 2^b$, then a loop over all capacities takes 2^b iterations.

The input encoding matters

Polynomial in the *value* of W can be exponential in its *bit length*.

This explains how a DP algorithm can coexist with NP-completeness of a decision version.

Kleinberg–Tardos, *Algorithm Design*, §6.4, p.271; §8.8, p.491.

Bellman–Ford: shortest paths as dynamic programming

Input: a directed graph, a source s, and edge lengths ℓ_{uv} that may be negative. Let $n = |V|$.

State: bound the number of edges

$D_i(v)$ is the minimum cost of an s–v walk using at most i edges. A walk may repeat vertices; an unreachable state has value ∞.

$$D_0(s) = 0, \qquad D_0(v) = \infty \quad (v \neq s).$$

Recurrence: omit the last edge, or choose it

$$D_i(v) = \min \left\{ D_{i-1}(v),\ \min_{(u,v)\in E} \big(D_{i-1}(u) + \ell_{uv} \big) \right\}.$$

Every dependency is in row $i - 1$; evaluate the rows in increasing i.

KT fixes a destination t and uses outgoing edges for v–t walks; here we fix a source s and use incoming edges for s–v walks.

Kleinberg–Tardos, *Algorithm Design*, §6.8, pp.290–294.

Bellman–Ford: repeated edge relaxation

BELLMAN-FORD(G, ℓ, s)

```
1  for each v ∈ V
2      d[v] = ∞
3      π[v] = NIL
4  d[s] = 0
5  for i = 1 to |V| − 1
6      for each edge (u, v) ∈ E
7          if d[u] < ∞ and d[v] > d[u] + ℓ_uv
8              d[v] = d[u] + ℓ_uv
9              π[v] = u
10 for each edge (u, v) ∈ E
11     if d[u] < ∞ and d[v] > d[u] + ℓ_uv
12         return "negative cycle reachable from source"
13 return (d, π)
```

Time: $O(nm)$ for $m \geq 1$; $O(n)$ initialization. **Auxiliary space:** $O(n)$.

Kleinberg–Tardos, *Algorithm Design*, §6.8, pp.295–296; §6.10, pp.301–303.

Bellman–Ford: why $n-1$ passes suffice

After pass i

Every finite $d[v]$ is the cost of an actual s–v walk, and

$$d[v] \le D_i(v).$$

In-place updates can propagate through several edges in one pass, so equality with the DP row is not required.

- Assume no negative cycle is reachable from s.
- Remove nonnegative cycles from a shortest walk. A simple shortest path has at most $n-1$ edges, so $D_{n-1}(v) = \delta(s, v)$.
- Labels are costs of actual walks, so they cannot be below $\delta(s, v)$.

Therefore, after $n-1$ passes,

$$\delta(s, v) \le d[v] \le D_{n-1}(v) = \delta(s, v).$$

Kleinberg–Tardos, *Algorithm Design*, §6.8, pp.293–296.

Bellman–Ford: detecting a reachable negative cycle

After $n-1$ passes, check whether some edge (u, v) satisfies

$$d[u] < \infty \qquad \text{and} \qquad d[v] > d[u] + \ell_{uv}.$$

- **No reachable negative cycle:** all reachable distances are correct, so no edge can still relax.
- **A reachable negative cycle exists:** its vertices have finite labels. If no edge of the cycle could relax, summing $d[v] \le d[u] + \ell_{uv}$ around it would imply $0 \le \sum \ell_{uv}$, a contradiction.

Reachability matters

This test detects only negative cycles reachable from s. To detect a negative cycle anywhere, add a new source with a zero-weight edge to every vertex.

Kleinberg–Tardos, *Algorithm Design*, §6.8, pp.293–296; §6.10, pp.301–303.

Tree decomposition: a tree of overlapping bags

For an undirected graph $G = (V, E)$, a tree decomposition is a tree T with a **bag** $B_t \subseteq V$ for each node t of T.

Three requirements

1. **Node coverage:** every vertex belongs to at least one bag: $\bigcup_{t \in V(T)} B_t = V$.
2. **Edge coverage:** for each $(u, v) \in E$, some bag contains both u and v.
3. **Coherence / running intersection:** for each vertex v, the nodes t with $v \in B_t$ induce a connected subtree of T.

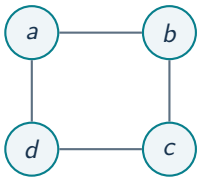
Equivalently, if v occurs in two bags, it occurs in every bag on the path between them.

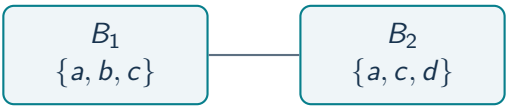
T is the *decomposition tree*; its nodes are bags, not individual vertices of G.

Kleinberg–Tardos, *Algorithm Design*, §10.4, pp.574–575.

Treewidth: minimize the largest bag, minus one

$$\text{width}(T, \{B_t\}) = \max_t |B_t| - 1, \qquad \text{tw}(G) = \min_{(T, \{B_t\})} \text{width}(T, \{B_t\}).$$





Largest bag size 3: this decomposition has width 2.

- A tree with at least one edge has treewidth 1.
- A width-k decomposition has bags of size at most $k + 1$.
- A bag need not be a clique: a and c above are not adjacent.

Kleinberg–Tardos, *Algorithm Design*, §10.4, pp.575–579.

Independent Set on a tree decomposition: state and boundary

Root the supplied width-k decomposition at r. Let T_t be the subtree at t and $U_t = \bigcup_{x \in V(T_t)} B_x$.

State: selected vertices on the current bag

For $S \subseteq B_t$, define

$$F_t(S) = \max\{|I| : I \subseteq U_t \text{ is independent}, I \cap B_t = S\}.$$

Set $F_t(S) = -\infty$ when S is not independent.

- Child regions $U_c \setminus B_t$ are disjoint, with no edges between them. Fixing the choices on B_t makes their subproblems independent.
- At most 2^{k+1} states per bag.
- **Leaf:** $F_t(S) = |S|$ for independent S.
- **Answer:** $\alpha(G) = \max_{S \subseteq B_r} F_r(S)$.

Kleinberg–Tardos, *Algorithm Design*, §10.4, pp.575–577, pp.580–581.

Independent Set: combine compatible child states

Fix an independent set $S \subseteq B_t$. A child state $R \subseteq B_c$ is compatible precisely when

$$R \cap B_t = S \cap B_c.$$

Recurrence

$$F_t(S) = |S| + \sum_{c \text{ child of } t} \max_{\substack{R \subseteq B_c \text{ independent} \\ R \cap B_t = S \cap B_c}} (F_c(R) - |S \cap B_c|).$$

- Agreement on the overlap makes the child solutions consistent.
- Separation ensures their union introduces no edge between selected vertices in different child regions.
- Subtract $|S \cap B_c|$: these selected boundary vertices are already counted by $|S|$.

Kleinberg–Tardos, *Algorithm Design*, §10.4, pp.582–583.

Independent Set: DP algorithm and complexity

TREE-DECOMPOSITION-IS($G, T, \{B_t\}$)

```
root T at r
for each node t of T in postorder
    for each independent set S ⊆ B_t
        F_t[S] = |S|
        for each child c of t
            best = −∞
            for each independent set R ⊆ B_c
                if R ∩ B_t = S ∩ B_c
                    best = max{best, F_c[R] − |S ∩ B_c|}
            F_t[S] = F_t[S] + best
return max_{S⊆B_r independent} F_r[S]
```

q bags, width k.

At most 2^{k+1} states per bag and 4^{k+1} state pairs per tree edge.

Direct implementation

$O\big(q(k+1)4^{k+1}\big)$ time,

$O\big(q2^{k+1}\big)$ table space.

Optimization: group child states by their selection on the bag intersection to obtain $O(q\,\mathrm{poly}(k)\,2^{k+1})$ time.
With a nonredundant decomposition, $q \le n$: fixed k gives $O(n)$ time. The decomposition is supplied; computing treewidth is NP-hard. Store child choices to reconstruct a solution.

Kleinberg–Tardos, *Algorithm Design*, §10.4, pp.579–584; §10.5, pp.584–585.

Exercise 6: Interleavings of repeating signals

For a nonempty binary string x, call any prefix of x^k for some $k \geq 0$ a **repetition** of x.

A string s is an **interleaving** of x and y if its positions can be partitioned into two subsequences: one a repetition of x, the other a repetition of y. Every position belongs to exactly one subsequence. Either subsequence may end partway through a copy of its repeating string.

Design a polynomial-time algorithm deciding whether given strings s, x, y satisfy this condition. Give the recurrence, pseudocode, correctness proof, and running-time bound.

Kleinberg–Tardos, *Algorithm Design*, Ch.6, Ex.19, p.329.

Solution 6: Interleavings of repeating signals

Let $p = |x|$, $q = |y|$, $N = |s|$. State $D[i, j]$ means that the first $i + j$ positions can be assigned to the first i symbols of x^∞ and first j symbols of y^∞.

$$D[i,j] = \big(i > 0 \wedge D[i-1,j] \wedge s_{i+j} = x_{1+((i-1) \bmod p)}\big) \\ \vee \big(j > 0 \wedge D[i,j-1] \wedge s_{i+j} = y_{1+((j-1) \bmod q)}\big).$$

```
initialize all states to false; D[0,0] ← true
for ℓ = 1, ..., N
    for i = 0, ..., ℓ
        j ← ℓ − i
        compute D[i,j] using the recurrence
return ⋁_{i=0}^{N} D[i, N − i]
```

Correctness. The last position belongs to exactly one signal; removing it gives exactly one of the two transitions. Induct on $i + j$. No divisibility condition is imposed, so partial copies are allowed. Time and space $O(N^2)$; rolling diagonals reduce space to $O(N)$.

Kleinberg–Tardos, *Algorithm Design*, Ch.6, Ex.19, p.329.

Exercise 7: Output a negative cycle

A weighted directed graph is known to contain a negative-weight cycle, but no source vertex or cycle is supplied. Give an efficient algorithm that lists the vertices of one negative-weight cycle in cyclic order, handling a cycle in any component of the graph. Prove that the reported vertices form a directed cycle with negative total weight, and analyze the running time. Reporting only that a negative cycle exists is not sufficient.

CLRS, *Introduction to Algorithms*, 3rd ed., Ex.24.1-6, p.655.

Solution 7: Output a negative cycle

```
1  n ← |V|; all d[v] ← 0, π[v] ← NIL
2  for i = 1, ..., n
3      x ← NIL
4      for each edge (u, v)
5          if d[v] > d[u] + w(u, v)
6              d[v] ← d[u] + w(u, v)
7              π[v] ← u; x ← v
8  if x = NIL: return "no negative cycle"
9  y ← x
10 repeat n times: y ← π[y]
11 C ← [y]; z ← π[y]
12 while z ≠ y
13     append z to C; z ← π[z]
14 return reverse(C), in cyclic order
```

All-zero initialization acts as a super-source reaching every component.

An update in pass n is a Bellman–Ford witness: following n predecessors reaches a cycle. Predecessors traverse its edges backward, so reverse the list.

For each final predecessor edge $u \to v$, $w(u, v) \leq d[v] - d[u]$. At least one inequality on the cycle is strict: take the successor of the vertex updated last. Summing gives $w(C) < 0$.

Time $O(n|E|)$; extra space $O(n)$.

CLRS, *Introduction to Algorithms*, 3rd ed., Ex.24.1-6, p.655.

Exercise 8: Construct a width-2 decomposition

A **triangulated cycle graph** is the boundary and triangulation of a convex polygon: all vertices lie on the polygon boundary, the diagonals do not cross, and every bounded face is a triangle.

1. Prove that every such graph has a tree decomposition of width at most 2.
2. Give an efficient algorithm constructing the decomposition from the triangulated polygon.

Your proof must verify node coverage, edge coverage, coherence, and the width bound. Analyze the construction's running time.

Kleinberg–Tardos, *Algorithm Design*, Ch.10, Ex.4, pp.596–597.

Solution 8: Construct a width-2 decomposition

```
create one bag containing the three vertices of each triangular face
connect two bags iff their faces share an internal diagonal
return the bag tree and its vertex sets
```

The bag graph is a tree. It is the weak dual of the triangulation: it is connected and has $n-2$ nodes and $n-3$ edges. Connectivity also follows by splitting the polygon recursively along diagonals.

- **Node coverage:** every polygon vertex belongs to a triangle.
- **Edge coverage:** every boundary edge or diagonal belongs to a triangle.
- **Coherence:** triangles incident to a fixed vertex form a consecutive fan; their bags form a connected path in the weak dual.

Every bag has size 3, so the width is 2. Enumerate faces and pair their diagonal incidences in $O(n)$ time and space using the supplied polygon embedding.

Kleinberg–Tardos, *Algorithm Design*, Ch.10, Ex.4, pp.596–597.

Exercise 9: Minimum-cost Dominating Set

In an undirected graph, a set S **dominates** every vertex outside S if each such vertex has a neighbor in S. Each vertex has a cost $c(v)$; minimize $\sum_{v \in S} c(v)$.

1. Give a polynomial-time algorithm when the graph is a tree.
2. Give a polynomial-time algorithm when a width-2 tree decomposition is supplied.

For each algorithm, define the states, derive the transitions, and prove correctness. State what information the boundary must retain.

Kleinberg–Tardos, *Algorithm Design*, Ch.10, Ex.5(a)–(b), p.597.

Solution 9: Minimum-cost Dominating Set

Assume the supplied decomposition has $O(n)$ bags. Convert to nice bags: empty leaf/root, introduce/forget one vertex, or join equal bags. For a tree, use bags $\{v, \text{parent}(v)\}$ and a singleton root (width 1).

A bag vertex has status **IN** (selected), **DOM** (unselected, already dominated in the processed subgraph), or **NEED** (unselected, still undominated). Store minimum cost per status vector, including costs of selected bag vertices; infeasible states cost ∞.

- **Leaf:** the empty state costs 0.
- **Introduce** v**:** either select it, add $c(v)$ and change adjacent NEED vertices to DOM; or leave it unselected, marking it DOM iff a bag neighbor is IN, otherwise NEED.
- **Forget** v**:** reject NEED; otherwise drop its status and minimize over child states.
- **Join:** IN vertices must agree in both children. An unselected vertex is DOM iff either child marks it DOM. Add child costs, subtracting $\sum_{v \in B:\, v \text{ IN}} c(v)$ once.

Correctness. Only bag vertices interact with the unprocessed graph. Forgetting checks domination before that opportunity ends; joins combine both regions. Return the empty-root value.

There are 3^{k+1} states and $O(9^{k+1})$ join pairs per bag. For width 1 or 2, the DP takes $O(n)$ time on $O(n)$ nice bags. Store choices to recover a set.

Optimization: recording domination requirements rather than exact statuses allows join time $O(\text{poly}(k)\, 4^{k+1})$ per bag, by assigning each required vertex to one child.

Kleinberg–Tardos, *Algorithm Design*, Ch.10, Ex.5(a)–(b), p.597.

Decision problems and optimization problems

A **decision problem** has a yes/no answer. Identify it with the set of binary strings whose answer is yes.

Problem	Optimization version	Decision version
Independent Set	Find the largest independent set.	Is there one of size at least k?
Vertex Cover	Find the smallest vertex cover.	Is there one of size at most k?
Subset Sum	Maximize the sum subject to sum $\leq W$.	Is some subset's sum exactly W?

The formal classes P, NP, and NP-complete here refer to **decision problems**.

Kleinberg–Tardos, *Algorithm Design*, §8.1, pp.454–455; §8.3, p.463; §8.8, p.491.

P and NP

P

Decision problems solvable by a polynomial-time algorithm: time polynomial in input length $|s|$.

> "We define NP to be the set of all problems for which there exists an efficient certifier."

A polynomial-time certifier B must satisfy, for some polynomial p,

$$s \in X \quad \Longleftrightarrow \quad \exists t : \ |t| \leq p(|s|) \ \text{ and } \ B(s,t) = \text{yes}.$$

- t is a polynomial-length certificate for a yes instance.
- Every no instance must reject every certificate of the allowed length.
- $\mathrm{P} \subseteq \mathrm{NP}$. NP means *nondeterministic polynomial time*.

Kleinberg–Tardos, *Algorithm Design*, §8.3, pp.463–465.

NP membership: what is the certificate?

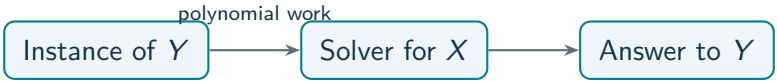
Problem	Certificate	Verification
SAT / 3-SAT	Truth values of variables	Evaluate every clause.
Independent Set	A list of vertices	Check at least k distinct vertices and no internal edge.
Vertex Cover	A list of vertices	Check at most k vertices and every edge is covered.
Subset Sum	Indices of selected items	Add the selected integers and compare with W.

Both certificate length and verification time must be polynomial in the *encoded input length*.

Kleinberg–Tardos, *Algorithm Design*, §8.3, pp.464–465; §8.8, p.492.

Polynomial-time reduction: the direction matters

Textbook convention: $Y \leq_P X$ if arbitrary instances of Y can be solved using polynomially many ordinary steps and polynomially many calls to a black box for X.



Interpretation and consequence

X is at least as hard as Y.
If $Y \leq_P X$ and $X \in \mathrm{P}$, then $Y \in \mathrm{P}$.

To transfer hardness *to* X, reduce a known hard problem *to* X.

Kleinberg–Tardos, *Algorithm Design*, §8.1, pp.452–453.

Polynomial-time reductions: a common form

Most textbook proofs construct a **single equivalent instance**:

$$f : \text{instances of } Y \longrightarrow \text{instances of } X, \qquad s \in Y \iff f(s) \in X.$$

- f must be computable in polynomial time.
- Prove both directions of the equivalence.
- The target instance must have polynomial encoding length.

Terminology in the textbook

Single-instance transformations: **Karp reductions**.
General polynomial black-box reductions: **Cook / Turing reductions**.

The concrete reductions in the following slides are single-instance transformations.

Kleinberg–Tardos, *Algorithm Design*, §8.1, p.458; §8.4, p.473.

NP-hard and NP-complete

> "We use the term NP-hard to mean 'at least as hard as an NP-complete problem."'

NP-hard

Every $Y \in \mathrm{NP}$ reduces to X:

$$\forall Y \in \mathrm{NP}, \quad Y \leq_P X.$$

The problem X need not be in NP or be a decision problem.

NP-complete

A decision problem X with $X \in \mathrm{NP}$ that is NP-hard.

Finding a maximum independent set is NP-hard; asking whether one of size at least k exists is NP-complete.

An NP-complete problem is polynomial-time solvable **if and only if** $\mathrm{P} = \mathrm{NP}$.

Kleinberg–Tardos, *Algorithm Design*, §8.4, p.466; Chapter 10, p.553, footnote 1.

A common template for NP-completeness proofs

Goal: prove X is NP-complete.

1. Show $X \in \mathrm{NP}$: give a certificate and polynomial-time verifier.
2. Choose a known NP-complete problem Y.
3. Transform any instance of Y into an instance of X.
4. Bound the construction time and output size by polynomials.
5. Prove source yes $\Rightarrow$ target yes.
6. Prove target yes $\Rightarrow$ source yes.

Correct direction

Prove $Y \leq_P X$. A reduction $X \leq_P Y$ alone does not prove X is hard.

Kleinberg–Tardos, *Algorithm Design*, §8.4, pp.470–473.

SAT and 3-SAT

A **literal** is a variable x or its negation $\neg x$.
A **clause** is a disjunction of literals; the formula is a conjunction of clauses.

"Given a set of clauses $C_1, \ldots, C_k$ over a set of variables $X = \{x_1, \ldots, x_n\}$, does there exist a satisfying truth assignment?"

3-SAT

Every clause contains exactly three terms corresponding to distinct variables, using the textbook convention.

$$(x \vee y \vee z) \ \wedge \ (\neg x \vee y \vee \neg z).$$

SAT and 3-SAT are NP-complete. A truth assignment is the certificate.

Kleinberg–Tardos, *Algorithm Design*, §8.2, pp.459–460; §8.4, pp.468–472.

Some common NP-complete problems

Problem	Decision question
Independent Set (IS)	At least k vertices, with no edge between selected vertices?
Vertex Cover (VC)	At most k vertices, touching every edge?
Set Cover	At most k given subsets whose union is the whole universe?
Set Packing	At least k given subsets that are pairwise disjoint?
3DM	n disjoint triples covering three disjoint sets of n elements each?
Subset Sum	A subset of the given integers whose sum is exactly W?
Hamiltonian Cycle	A directed cycle visiting every vertex exactly once before returning to its start?
3-Partition (strongly NP-complete)	Given $3m$ positive integers with $\sum_i a_i = mB$ and $B/4 < a_i < B/2$, can they be partitioned into m triples, each summing to B?

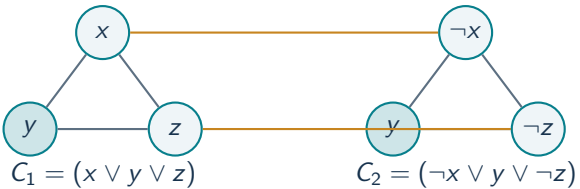
Strongly NP-complete: remains NP-complete even when the input integers are encoded in unary. Thus hardness does not rely on large numbers having short binary encodings.

Kleinberg–Tardos, *Algorithm Design*, §8.1–8.6, pp.454–485; §8.8, pp.491–492.
3-Partition: *Algorithmic Lower Bounds*, §5.3.2.

3-SAT $\leq_P$ Independent Set: the construction

For a formula with m clauses:

- Create one triangle per clause, with one vertex per literal occurrence.
- Join occurrences in different clauses if their literals contradict.
- Ask whether the resulting graph has an independent set of size m.



Kleinberg–Tardos, *Algorithm Design*, §8.2, pp.460–462.

3-SAT $\leq_P$ Independent Set: correctness

Satisfying assignment $\Rightarrow$ independent set

Pick one true literal occurrence from each clause.
No two selected literals contradict, and only one comes from each triangle. The m selected vertices form an independent set.

Independent set $\Rightarrow$ satisfying assignment

Each triangle contributes at most one vertex. A set of size at least m must contain exactly one from every triangle.
No selected literals contradict, so assign them all true and assign the remaining variables arbitrarily. Every clause is satisfied.

Size: $3m$ vertices and $O(m^2)$ edges. Construction is polynomial.

Kleinberg–Tardos, *Algorithm Design*, §8.2, pp.460–462.

Independent Set $\leftrightarrow$ Vertex Cover

"Let $G = (V, E)$ be a graph. Then S is an independent set if and only if its complement $V - S$ is a vertex cover."

- If S is independent, no edge has both ends in S. Every edge therefore has an end in $V \setminus S$.
- If $V \setminus S$ is a cover, no edge can have both ends in S.

The instance transformation

$$(G, k) \in \mathrm{IS} \quad \Longleftrightarrow \quad (G, |V| - k) \in \mathrm{VC}.$$

Keep the graph; complement the size threshold. The same idea gives a reduction in either direction.

Kleinberg–Tardos, *Algorithm Design*, §8.1, pp.455–456.

Vertex Cover $\leq_P$ Set Cover

Vertex Cover instance

- Graph $G = (V, E)$.
- Budget k vertices.

Set Cover instance

- Universe $U = E$.
- One set S_v for each $v \in V$.
- $S_v = \{e \in E : e \text{ touches } v\}$.
- Same budget k.

Equivalence

Chosen vertices cover every edge
$\iff$ their incident-edge sets cover every element of U.

Construction size: $|V|$ sets and $2|E|$ total edge memberships for a simple undirected graph.

Kleinberg–Tardos, *Algorithm Design*, §8.1, pp.456–458.

3-Dimensional Matching (3DM)

"Given disjoint sets X, Y, and Z, each of size n, and given a set $T \subseteq X \times Y \times Z$ of ordered triples, does there exist a set of n triples in T so that each element of $X \cup Y \cup Z$ is contained in exactly one of these triples?"

Perfect three-dimensional matching

Choose $M \subseteq T$ such that

$$|M| = n, \qquad \text{no two chosen triples share an element.}$$

- Each triple contains one element from each of X, Y, Z.
- **Certificate:** the chosen triples. Check membership in T and that every element is covered exactly once in polynomial time.
- 3DM is NP-complete; ordinary bipartite matching is polynomial-time solvable.

Kleinberg–Tardos, *Algorithm Design*, §8.6, pp.481–482.

3-SAT $\leq_P$ 3DM: the reduction idea

Variable gadgets. Two alternating ways to cover each variable core encode true or false and leave corresponding literal tips available.

Clause gadgets. Each clause pair must use an available tip from one of its literals, enforcing that at least one literal is true.

Cleanup gadgets. Match all remaining tips so every element is covered exactly once.

Correctness outline

A satisfying assignment determines compatible variable and clause choices; cleanup completes the matching.
A perfect matching forces a consistent truth choice in each variable gadget and a true literal in every clause.

The construction has polynomial size and partitions all elements into three equally sized coordinate sets.

Kleinberg–Tardos, *Algorithm Design*, §8.6, pp.482–485.

3DM connects exact covering and disjoint packing

Let $U = X \cup Y \cup Z$ and view each allowed triple as a three-element subset of U.

The same instance has three equivalent descriptions

- A perfect three-dimensional matching of size n.
- A Set Packing of at least n triples.
- A Set Cover of at most n triples.

Since $|U| = 3n$ and each set has size 3, packing n disjoint triples or covering U with n triples forces exact coverage.

$$\text{3DM} \leq_P \textsc{Set Packing}, \qquad \text{3DM} \leq_P \textsc{Set Cover}.$$

These outgoing reductions alone do not prove 3DM hard; its hardness comes from $\text{3-SAT} \leq_P \text{3DM}$.

Kleinberg–Tardos, *Algorithm Design*, §8.6, pp.481–482.

Exercise 10: Multiple Interval Scheduling

Each job requests a finite set of time intervals. Accepting a job reserves *all* its intervals. A single processor can execute at most one job at any instant.

Decision problem

Can at least k jobs be accepted so that no interval of one accepted job overlaps an interval of another accepted job?

Prove NP-completeness. Your construction must preserve exactly the compatibility relations needed for both directions of the reduction.

Kleinberg–Tardos, *Algorithm Design*, Ch.8, Ex.14, p.512.

Solution 10: Multiple Interval Scheduling

In NP. A certificate lists at least k accepted jobs. Check all cross-job interval pairs for overlap in polynomial time.

Reduction from Independent Set. Given (G, k):

- Allocate a separate time block I_e to each edge e, and a private block P_v to each vertex v. All blocks are pairwise disjoint, e.g. $[3j, 3j + 1]$.
- Create job J_v requesting P_v and every I_e with e incident to v.
- Keep the target k unchanged.

Two distinct jobs share a block exactly when their vertices are adjacent:

$$\{J_v : v \in S\} \text{ is feasible} \quad \Longleftrightarrow \quad S \text{ is an independent set in } G.$$

The private blocks also handle isolated vertices. There are $|V| + 2|E|$ interval occurrences and polynomial-size integer endpoints. This is a polynomial reduction, proving NP-completeness.

Kleinberg–Tardos, *Algorithm Design*, Ch.8, Ex.14, p.512.

Exercise 11: A simple cycle of exactly zero weight

A directed graph has integer edge weights, which may be positive or negative.

Zero-Weight-Cycle

Does the graph contain a *simple* directed cycle whose edge weights sum to exactly 0?

1. Prove that this decision problem is in NP.
2. Prove NP-hardness by a polynomial-time reduction, establishing both directions of the instance equivalence and bounding the construction size.

Kleinberg–Tardos, *Algorithm Design*, Ch.8, Ex.17, p.513.

Solution 11: A simple cycle of exactly zero weight

In NP. List the distinct cycle vertices; check its directed edges and sum their binary-encoded weights in polynomial time.

Reduction from Subset Sum. For positive integers $a_1, \ldots, a_n$ and target $W > 0$, create vertices $p_0, \ldots, p_n$. At stage i, add two fresh branch vertices t_i, f_i and edges

$$p_{i-1} \xrightarrow{a_i} t_i \xrightarrow{0} p_i, \qquad p_{i-1} \xrightarrow{0} f_i \xrightarrow{0} p_i.$$

Add one return edge $p_n \to p_0$ of weight $-W$.

Without the return edge the graph is a DAG. Every directed cycle therefore uses that edge and exactly one branch at every stage. Such a cycle is simple, and

$$w(C) = \sum_{i:\, C \text{ takes } t_i} a_i - W.$$

A subset sums to W iff the graph has a zero-weight simple cycle. The graph has $O(n)$ vertices and edges, preserves the input numbers' bit lengths, and uses no parallel edges. Hence the problem is NP-complete.

Kleinberg–Tardos, *Algorithm Design*, Ch.8, Ex.17, p.513.

Exercise 12: From a decision oracle to an independent set

A black box answers whether an undirected graph contains an independent set of size at least k. Count each query as one operation.

1. Use polynomially many queries and polynomial additional work to output a maximum independent set. Prove that the returned vertices are valid and optimal.
2. Without this black box, solve Independent Set efficiently for graphs in which every vertex has degree 2. Prove correctness and analyze the running time.

The first part asks for an actual set of vertices, not only its cardinality.

CLRS, *Introduction to Algorithms*, 3rd ed., Problem 34-1(b)–(c), pp.1101–1102.

Solution 12: From decision to an independent set

First binary-search the optimum size k using $O(\log n)$ oracle queries.

```
1  H ← G;  I ← ∅
2  while k > 0
3      choose any v ∈ V(H)
4      if ORACLE(H − v, k) = yes
5          H ← H − v
6      else
7          I ← I ∪ {v}
8          H ← H − N_H[v]
9          k ← k − 1
10 return I
```

$N_H[v]$ is the closed neighborhood of v. Each iteration deletes at least one vertex: $O(n)$ queries total, with polynomial additional work.

Invariant. H has an independent set of size at least k. A yes answer preserves it. A no answer means every such set contains v; deleting its closed neighborhood leaves a solution of size at least $k - 1$.

Chosen vertices stay independent, and the number chosen equals the original optimum.

Degree 2. Each connected component is a cycle. On a cycle of length ℓ, take $\lfloor \ell/2 \rfloor$ alternating vertices, avoiding wraparound for odd ℓ. No independent set is larger, since $2|I| \leq \ell$. Combine the components in $O(n + m)$ time.

CLRS, *Introduction to Algorithms*, 3rd ed., Problem 34-1(b)–(c), pp.1101–1102.

Sources and reading guide

Jon Kleinberg and Éva Tardos, *Algorithm Design*

- Stable matching: §1.1, pp.1–12; implementation: §2.3, pp.42–47.
- Strong connectivity: §3.5, pp.97–99.
- Dijkstra / MST: §4.4–4.6, pp.137–157.
- Greedy, interval scheduling, exchange: pp.115–130.
- Huffman coding: §4.8, pp.161–175.
- DP: §6.1–6.4, pp.251–272; Bellman–Ford: §6.8, pp.290–296; negative cycles: §6.10, pp.301–303.
- Tree decomposition / treewidth / Independent Set: §10.4, pp.572–584.
- Reductions, NP, NP-completeness: §8.1–8.4, pp.452–473.
- Further NPC problems, including 3DM: §8.5–8.8, pp.474–496.
- NP-hard terminology: p.553, footnote 1.

Cormen, Leiserson, Rivest, Stein, *Introduction to Algorithms*, 3rd ed.

- BFS: §22.2, pp.594–602; DFS: §22.3, pp.603–612.

CS161, Winter 2011 Handout #16, *Notes on Strongly Connected Components*, pp.1–5.

Thank you

Thank you for your attention!

Thanks to the course instructors, teaching assistants, textbook authors, and everyone attending this review.

These review notes may contain errors or omissions. Please consult the course materials and cited references for clarification.

Corrections and suggestions are welcome.